

Lecture 1: Introduction and LU Factorization

Math 5610/6610

C. Mou

Department of Mathematics and Statistics
Utah State University

Fall 2026

Course Information

- **Course:** MATH 5610/6610, Introduction to Numerical Analysis, Fall 2026
- **Instructor:** Dr. Changhong Mou
- **Office:** Geology 417B
- **Email:** changhong.mou@usu.edu
- **Office Hours:** Mon 2:00–3:00 p.m., R 3:00–4:00 p.m., or by appointment
- **Lectures:** MWF 10:30–11:20 a.m., Old Main 203
(Aug 31 – Dec 11, 2026)
- **Course page:** <https://chhmou.github.io/math5610/>
 - lecture notes, slides, homework and solutions, and final project information are posted here throughout the semester
 - also linked from the Canvas syllabus

Grading

- **Grading scheme:**

- ▶ Homework: 40% (4 assignments, 10% each)
- ▶ Midterm Exam: 20%
- ▶ Final Project: 40%

- **Homework:** combines theoretical problems with programming exercises in MATLAB or Python

- **Midterm:** one in-class exam around the middle of the semester

- **Final Project** (in lieu of a final exam):

- ▶ implementation and analysis of a numerical method related to the course material
- ▶ short written report and a brief in-class presentation during the last week of classes or the final examination period
- ▶ project topics must be approved by the instructor

Textbooks, Software, and Important Dates

- **Suggested textbooks** (no required textbook):
 - Lecture notes
 - Trefethen & Bau, *Numerical Linear Algebra*
 - Stoer & Bulirsch, *An Introduction to Numerical Analysis*, 2nd ed.
- **Software:** MATLAB is recommended
- **No class:**
 - Mon, Sep 7 (Labor Day); Mon, Sep 21 (instructor travel)
 - Fri, Oct 9 (Fall Break); Wed, Nov 18 (instructor travel)
 - Nov 25–27 (Thanksgiving Break)
- **Other dates:** Dec 7–11 is No-Test Week; last day of classes Fri, Dec 11; final examinations Dec 14–18

Course Overview

- Graduate-level course on the theory and practice of numerical algorithms; emphasis on rigorous error analysis alongside efficient implementation

Course Overview

- Graduate-level course on the theory and practice of numerical algorithms; emphasis on rigorous error analysis alongside efficient implementation
- Main topics:
 - ▶ direct methods for linear systems: LU factorization with pivoting
 - ▶ Newton's method for nonlinear systems and its convergence theory
 - ▶ floating-point arithmetic, conditioning, and (backward) stability
 - ▶ polynomial and spline interpolation, including B-splines
 - ▶ numerical integration: Newton–Cotes, Gaussian, and adaptive quadrature
 - ▶ eigenvalue computation: unitary similarity and the QR algorithm

Course Overview

- Graduate-level course on the theory and practice of numerical algorithms; emphasis on rigorous error analysis alongside efficient implementation
- Main topics:
 - ▶ direct methods for linear systems: LU factorization with pivoting
 - ▶ Newton's method for nonlinear systems and its convergence theory
 - ▶ floating-point arithmetic, conditioning, and (backward) stability
 - ▶ polynomial and spline interpolation, including B-splines
 - ▶ numerical integration: Newton–Cotes, Gaussian, and adaptive quadrature
 - ▶ eigenvalue computation: unitary similarity and the QR algorithm
- **Prerequisites:** linear algebra and advanced calculus/introductory analysis; an undergraduate numerical methods course (e.g., MATH 4610) is recommended; programming experience in MATLAB, Python, or similar

Today's Goals

- Understand what numerical analysis studies and why exact answers are usually impossible

Today's Goals

- Understand what numerical analysis studies and why exact answers are usually impossible
- Distinguish the three sources of error: conditioning, stability, convergence

Today's Goals

- Understand what numerical analysis studies and why exact answers are usually impossible
- Distinguish the three sources of error: conditioning, stability, convergence
- See Gaussian elimination reinterpreted as a matrix factorization $A = LU$

Today's Goals

- Understand what numerical analysis studies and why exact answers are usually impossible
- Distinguish the three sources of error: conditioning, stability, convergence
- See Gaussian elimination reinterpreted as a matrix factorization $A = LU$
- Meet Frobenius matrices, the building blocks of elimination

What is Numerical Analysis?

Definition (Numerical Analysis)

Numerical analysis is the study of *algorithms* for the problems of *continuous mathematics*.
(Trefethen)

What is Numerical Analysis?

Definition (Numerical Analysis)

Numerical analysis is the study of *algorithms* for the problems of *continuous mathematics*.
(Trefethen)

Problem:

Input $\longrightarrow$ Output: Solution(s) or No Solution

What is Numerical Analysis?

Definition (Numerical Analysis)

Numerical analysis is the study of *algorithms* for the problems of *continuous mathematics*.
(Trefethen)

Problem:

Input $\longrightarrow$ Output: Solution(s) or No Solution

Numerical Methods:

(Approximate) Input $\longrightarrow$ (Approximate) Solution

General Remarks

- Almost no problem of continuous mathematics can be solved *exactly* on a computer:
 - inputs are measured or rounded before the computation begins
 - every arithmetic operation is itself inexact
 - many algorithms must be stopped after finitely many steps

General Remarks

- Almost no problem of continuous mathematics can be solved *exactly* on a computer:
 - inputs are measured or rounded before the computation begins
 - every arithmetic operation is itself inexact
 - many algorithms must be stopped after finitely many steps
- So a numerical method never reproduces the exact input–output map of the mathematical problem; the question is *how far* the computed answer is from the true one, and *which part of the process* is responsible for the gap

General Remarks

- Almost no problem of continuous mathematics can be solved *exactly* on a computer:
 - inputs are measured or rounded before the computation begins
 - every arithmetic operation is itself inexact
 - many algorithms must be stopped after finitely many steps
- So a numerical method never reproduces the exact input–output map of the mathematical problem; the question is *how far* the computed answer is from the true one, and *which part of the process* is responsible for the gap
- The subject organizes this gap into three recurring themes:
 - conditioning: sensitivity of the *problem*
 - stability: reliability of the *algorithm*
 - convergence: accuracy of *iteration* after finitely many steps

General Remarks

- Almost no problem of continuous mathematics can be solved *exactly* on a computer:
 - inputs are measured or rounded before the computation begins
 - every arithmetic operation is itself inexact
 - many algorithms must be stopped after finitely many steps
- So a numerical method never reproduces the exact input–output map of the mathematical problem; the question is *how far* the computed answer is from the true one, and *which part of the process* is responsible for the gap
- The subject organizes this gap into three recurring themes:
 - conditioning: sensitivity of the *problem*
 - stability: reliability of the *algorithm*
 - convergence: accuracy of *iteration* after finitely many steps
- These three themes guide the entire course

Sources of Errors

- ➊ **Approximate input errors:** cannot represent the data exactly (e.g., irrational numbers)

⇒ conditioning of the problem

These errors are *inherent to the problem*.

Sources of Errors

- 1 **Approximate input errors:** cannot represent the data exactly (e.g., irrational numbers)

⇒ conditioning of the problem

These errors are *inherent to the problem*.

- 2 **Rounding errors:**

⇒ stability of the algorithm

Sources of Errors

- ➊ **Approximate input errors:** cannot represent the data exactly (e.g., irrational numbers)

⇒ conditioning of the problem

These errors are *inherent to the problem*.

- ➋ **Rounding errors:**

⇒ stability of the algorithm

- ➌ **Approximation errors:** terminating the algorithm when the result is “close enough”

⇒ convergence

Two Model Problems

Example (Systems of linear equations)

$$A\mathbf{x} = \mathbf{b}, \quad A \in \mathbb{C}^{n \times n}, \mathbf{b} \in \mathbb{C}^n, A \text{ nonsingular}$$

- Input: $A, \mathbf{b}$
- Solution: $\mathbf{x} = A^{-1}\mathbf{b}$
- Obtained via LU factorization (= modern Gaussian elimination)

Two Model Problems

Example (Systems of linear equations)

$$A\mathbf{x} = \mathbf{b}, \quad A \in \mathbb{C}^{n \times n}, \mathbf{b} \in \mathbb{C}^n, A \text{ nonsingular}$$

- Input: $A, \mathbf{b}$
- Solution: $\mathbf{x} = A^{-1}\mathbf{b}$
- Obtained via LU factorization (= modern Gaussian elimination)

Example (Systems of nonlinear equations)

$$f(\mathbf{x}) = 0$$

- Input: $f : D \rightarrow \mathbb{R}^n, D \subset \mathbb{R}^n$
- Solutions: $\mathbf{x} \in D$ such that $f(\mathbf{x}) = 0$
- Special case: $f(\mathbf{x}) = A\mathbf{x} - \mathbf{b}$ recovers the linear problem
- Standard method: Newton's method

Remarks on the Two Model Problems

- Writing $\mathbf{x} = A^{-1}\mathbf{b}$ describes the *answer*, not the *method*: in practice we **never** form A^{-1}
 - forming A^{-1} costs more, and (as we will see) it is also less accurate than solving via factorization

Remarks on the Two Model Problems

- Writing $\mathbf{x} = A^{-1}\mathbf{b}$ describes the *answer*, not the *method*: in practice we **never** form A^{-1}
 - forming A^{-1} costs more, and (as we will see) it is also less accurate than solving via factorization
- The two problems behave very differently as algorithms:
 - $A\mathbf{x} = \mathbf{b}$: a *direct* method (LU) reaches the answer in finitely many steps – in exact arithmetic, exactly
 - $f(\mathbf{x}) = 0$: in general *no* finite algorithm gives the answer; Newton's method generates a sequence of approximations and must be *stopped* “when close enough”

Remarks on the Two Model Problems

- Writing $\mathbf{x} = A^{-1}\mathbf{b}$ describes the *answer*, not the *method*: in practice we **never** form A^{-1}
 - ▶ forming A^{-1} costs more, and (as we will see) it is also less accurate than solving via factorization
- The two problems behave very differently as algorithms:
 - ▶ $A\mathbf{x} = \mathbf{b}$: a *direct* method (LU) reaches the answer in finitely many steps – in exact arithmetic, exactly
 - ▶ $f(\mathbf{x}) = 0$: in general *no* finite algorithm gives the answer; Newton's method generates a sequence of approximations and must be *stopped* “when close enough”
- This split is where the three themes attach:
 - ▶ direct methods $\Rightarrow$ conditioning and stability are the issues
 - ▶ iterative methods $\Rightarrow$ convergence joins them

Remarks on the Two Model Problems

- Writing $\mathbf{x} = A^{-1}\mathbf{b}$ describes the *answer*, not the *method*: in practice we **never** form A^{-1}
 - forming A^{-1} costs more, and (as we will see) it is also less accurate than solving via factorization
- The two problems behave very differently as algorithms:
 - $A\mathbf{x} = \mathbf{b}$: a *direct* method (LU) reaches the answer in finitely many steps – in exact arithmetic, exactly
 - $f(\mathbf{x}) = 0$: in general *no* finite algorithm gives the answer; Newton's method generates a sequence of approximations and must be *stopped* “when close enough”
- This split is where the three themes attach:
 - direct methods $\Rightarrow$ conditioning and stability are the issues
 - iterative methods $\Rightarrow$ convergence joins them
- The linear problem sits inside the nonlinear one ($f(\mathbf{x}) = A\mathbf{x} - \mathbf{b}$), and Newton's method solves a linear system at every step – another reason to get $A\mathbf{x} = \mathbf{b}$ right first

LU Factorization: The Goal

Given: (nonsingular) $A \in \mathbb{C}^{n \times n}$, $\mathbf{b} \in \mathbb{C}^n$.

Goal: solve $A\mathbf{x} = \mathbf{b}$.

LU Factorization: The Goal

Given: (nonsingular) $A \in \mathbb{C}^{n \times n}$, $\mathbf{b} \in \mathbb{C}^n$.

Goal: solve $A\mathbf{x} = \mathbf{b}$.

Gaussian elimination:

$$A\mathbf{x} = \mathbf{b} \longleftrightarrow U\mathbf{x} = \mathbf{g}, \quad U = \begin{bmatrix} * & * & \dots & * \\ 0 & * & \dots & * \\ \vdots & \ddots & \ddots & \vdots \\ 0 & \dots & 0 & * \end{bmatrix}$$

i.e., U is *upper-triangular*.

LU Factorization: The Goal

Given: (nonsingular) $A \in \mathbb{C}^{n \times n}$, $\mathbf{b} \in \mathbb{C}^n$.

Goal: solve $A\mathbf{x} = \mathbf{b}$.

Gaussian elimination:

$$A\mathbf{x} = \mathbf{b} \longleftrightarrow U\mathbf{x} = \mathbf{g}, \quad U = \begin{bmatrix} * & * & \dots & * \\ 0 & * & \dots & * \\ \vdots & \ddots & \ddots & \vdots \\ 0 & \dots & 0 & * \end{bmatrix}$$

i.e., U is *upper-triangular*. How do we do this?

$$L_{n-1} \cdots L_2 L_1 A = U \iff A = \underbrace{L_1^{-1} \cdots L_{n-2}^{-1} L_{n-1}^{-1}}_L U$$

$$L = \begin{bmatrix} 1 & 0 & \dots & 0 \\ * & 1 & \dots & 0 \\ \vdots & \ddots & \ddots & \vdots \\ * & \dots & * & 1 \end{bmatrix}$$

i.e., L is *lower-triangular*.

LU Factorization: A 4×4 Example

Example (General LU factorization, $n = 4$)

$$A = \begin{bmatrix} * & * & * & * \\ * & * & * & * \\ * & * & * & * \\ * & * & * & * \end{bmatrix} \xrightarrow{L_1} \begin{bmatrix} * & * & * & * \\ 0 & \# & \# & \# \\ 0 & \# & \# & \# \\ 0 & \# & \# & \# \end{bmatrix} = L_1 A$$

LU Factorization: A 4×4 Example

Example (General LU factorization, $n = 4$)

$$\begin{aligned} A = \begin{bmatrix} * & * & * & * \\ * & * & * & * \\ * & * & * & * \\ * & * & * & * \end{bmatrix} &\xrightarrow{L_1} \begin{bmatrix} * & * & * & * \\ 0 & \# & \# & \# \\ 0 & \# & \# & \# \\ 0 & \# & \# & \# \end{bmatrix} = L_1 A \\ \xrightarrow{L_2} \begin{bmatrix} * & * & * & * \\ 0 & * & * & * \\ 0 & 0 & \# & \# \\ 0 & 0 & \# & \# \end{bmatrix} = L_2 L_1 A &\xrightarrow{L_3} \begin{bmatrix} * & * & * & * \\ 0 & * & * & * \\ 0 & 0 & * & * \\ 0 & 0 & 0 & \# \end{bmatrix} = L_3 L_2 L_1 A = U \end{aligned}$$

LU Factorization: A 4×4 Example

Example (General LU factorization, $n = 4$)

$$A = \begin{bmatrix} * & * & * & * \\ * & * & * & * \\ * & * & * & * \\ * & * & * & * \end{bmatrix} \xrightarrow{L_1} \begin{bmatrix} * & * & * & * \\ 0 & \# & \# & \# \\ 0 & \# & \# & \# \\ 0 & \# & \# & \# \end{bmatrix} = L_1 A$$

$$\xrightarrow{L_2} \begin{bmatrix} * & * & * & * \\ 0 & * & * & * \\ 0 & 0 & \# & \# \\ 0 & 0 & \# & \# \end{bmatrix} = L_2 L_1 A \quad \xrightarrow{L_3} \begin{bmatrix} * & * & * & * \\ 0 & * & * & * \\ 0 & 0 & * & * \\ 0 & 0 & 0 & \# \end{bmatrix} = L_3 L_2 L_1 A = U$$

*'s represent unchanged nonzero entries;

#'s represent changed nonzero entries.

The L_i are *Frobenius matrices*.

Reading the 4×4 Picture

Two features of the shape are worth noticing now:

- Each step works on a smaller trailing block than the one before: L_1 updates a 3×3 block, L_2 a 2×2 block, L_3 a single entry
 - this shrinking is why the total operation count comes out to $\mathcal{O}(n^3)$ rather than $\mathcal{O}(n^4)$ (counted precisely next lecture)

Reading the 4×4 Picture

Two features of the shape are worth noticing now:

- Each step works on a smaller trailing block than the one before: L_1 updates a 3×3 block, L_2 a 2×2 block, L_3 a single entry
 - this shrinking is why the total operation count comes out to $\mathcal{O}(n^3)$ rather than $\mathcal{O}(n^4)$ (counted precisely next lecture)
- Each step is multiplication by a matrix L_i of a very particular form: identity plus one column of subdiagonal entries
 - these are the *Frobenius matrices* defined next, and their special structure is what makes the factor L available for free

Frobenius Matrices

Definition (Frobenius matrix)

A Frobenius matrix is a square matrix with the following properties:

- all entries on the main diagonal are ones
- the entries below the main diagonal of *at most one column* are arbitrary
- every other entry is zero

Frobenius Matrices

Definition (Frobenius matrix)

A Frobenius matrix is a square matrix with the following properties:

- all entries on the main diagonal are ones
- the entries below the main diagonal of *at most one column* are arbitrary
- every other entry is zero

Frobenius matrices are *invertible*, and the inverse is obtained by negating the off-diagonal entries:

$$A = \begin{bmatrix} 1 & 0 & 0 & \cdots & 0 \\ 0 & 1 & 0 & \cdots & 0 \\ 0 & a_{32} & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & a_{n2} & 0 & \cdots & 1 \end{bmatrix}, \quad A^{-1} = \begin{bmatrix} 1 & 0 & 0 & \cdots & 0 \\ 0 & 1 & 0 & \cdots & 0 \\ 0 & -a_{32} & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & -a_{n2} & 0 & \cdots & 1 \end{bmatrix}$$

Filling in the Details: Why Frobenius Inverses Are So Simple I

Write a Frobenius matrix acting on column k as

$$L_k = I + c e_k^T,$$

where e_k is the k th standard basis vector and $c \in \mathbb{C}^n$ collects the subdiagonal entries (so $c_j = 0$ for $j \leq k$).

Key observation: the k th entry of c is zero, so

$$e_k^T c = c_k = 0.$$

Now multiply:

$$\begin{aligned}(I + c e_k^T)(I - c e_k^T) &= I - c e_k^T + c e_k^T - c \underbrace{(e_k^T c)}_{=0} e_k^T \\ &= I.\end{aligned}$$

So the inverse is obtained by *flipping the signs* of the subdiagonal entries – no computation needed. This is exactly what we saw in the definition: A^{-1} replaces a_{j2} by $-a_{j2}$.

Filling in the Details: Why Frobenius Inverses Are So Simple II

Why this matters for LU: the elimination matrices $L_1, L_2, \dots, L_{n-1}$ are all Frobenius, so

$$L = L_1^{-1} L_2^{-1} \cdots L_{n-1}^{-1}$$

is available *for free*: each factor just contributes its (sign-flipped) column of multipliers, and (as we will verify next lecture) the product assembles them into one unit lower-triangular matrix with no fill-in. Gaussian elimination therefore produces L and U simultaneously at no extra cost.

The Three Errors on One Example

Take $Ax = b$ solved by elimination on a computer:

- 1 **Conditioning:** the entries of A , b may come from measurements, or may not be exactly representable in binary. How much can the exact solution $A^{-1}b$ move when (A, b) move slightly? *Property of the problem.*
- 2 **Stability:** even with exact input, every arithmetic operation is rounded. Does the algorithm amplify these roundings? *Property of the algorithm.*
- 3 **Convergence:** direct methods like LU terminate exactly, but iterative methods (Newton, QR iteration, ...) must be stopped “when close enough.” How fast do they approach the answer?

The first half of this course builds the vocabulary to make all three precise.

Looking Ahead

- **Next:** constructing the L_i explicitly and the LU algorithm

Looking Ahead

- **Next:** constructing the L_i explicitly and the LU algorithm
- Pivoting: why and how ($PA = LU$)

Looking Ahead

- **Next:** constructing the L_i explicitly and the LU algorithm
- Pivoting: why and how ($PA = LU$)
- Operation counts and computational cost

Looking Ahead

- **Next:** constructing the L_i explicitly and the LU algorithm
- Pivoting: why and how ($PA = LU$)
- Operation counts and computational cost
- Conditioning and stability of Gaussian elimination